

Rapport

Projet modélisation d'une foule



**ECOLE NATIONALE SUPERIEURE
DE MECANIQUE ET DES MICROTECHNIQUES**

BASSENE Jeannette

DAMBLEVE Florian

SOMMAIRE

Introduction	1
Présentation générale du projet	1
Diagrammes UML	1
Fonctionnement	2
Instance d'exécution	3
Algorithmes utilisés	5
Tests	8
Critiques et perspectives.....	9
Critiques.....	9
Perspectives.....	9
Conclusion	9
Table des illustrations.....	9

Introduction

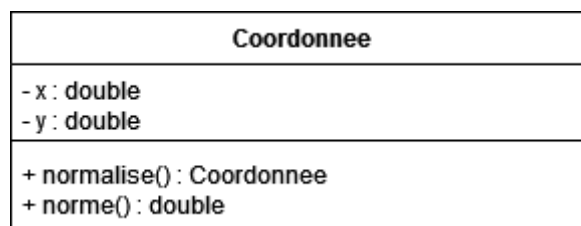
Création d'un programme permettant de simuler le mouvement d'une foule dans différente situation en partant de différentes classes. Possibilité de changer le nombre d'agents ainsi que leur place en début de simulation, mais aussi leur couleur et leur vitesse. Possibilité de changer la place de la sortie, la taille de la fenêtre de simulation, l'environnement où les agents vont évoluer. De plus, une étude sur la collision entre les agents, avec une version blocage puis une version d'évitement, a été réalisé.

Présentation générale du projet

Diagrammes UML

Pour ce projet, différentes classes ont d'abord été réalisée :

- Une classe Coordonnée permettant d'assigner des coordonnées réelles ainsi que de calculer la distance euclidienne entre deux points et dessiner la fenêtre de simulation,



- Une classe Agent assignant un cap, une vitesse, une position, un objectif, une couleur et un rayon à un point. Cette classe inclue une méthode permettant à l'agent de se déplacer,

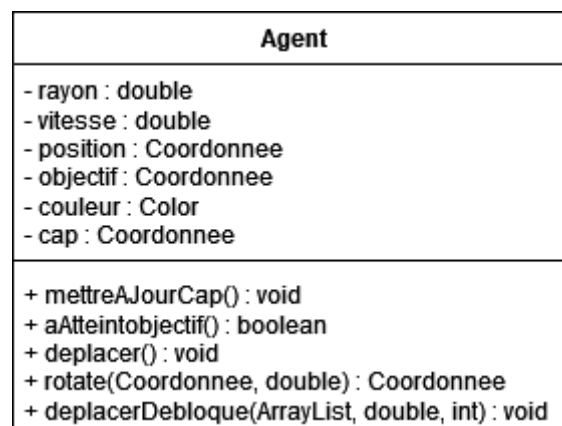
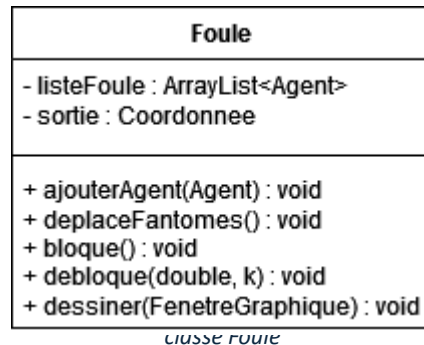


Figure 2 : Diagramme UML de la classe Agent

- Une classe Foule pour créer une liste d'agent et leur assigner une sortie. Une méthode permettant d'arrêter les agents en cas de collision et une autre pour qu'ils s'évitent ainsi qu'une méthode pour faire disparaître les agents une fois qu'ils sont à la sortie,



- On a donc une relation entre ces différentes classes,

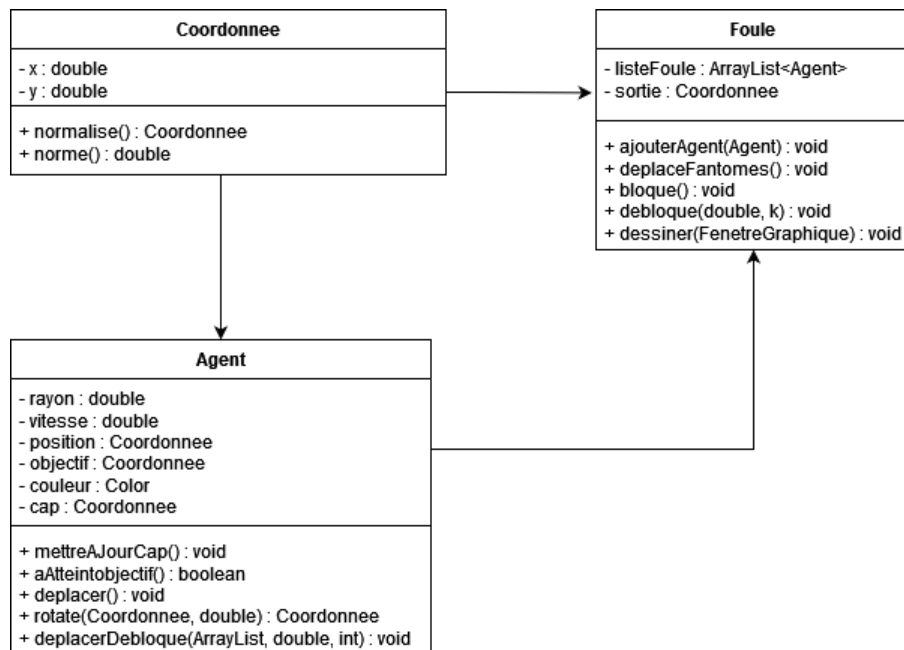


Figure 4 : Diagramme UML de l'ensemble des classes

Pour que le programme puisse se lancer, il faut, dans un premier temps, créer une fenêtre graphique à l'aide de la méthode mise à disposition. On lui affecte un nom ainsi qu'une longueur et une largeur.

Ensuite, on doit créer une sortie à partir de la classe coordonnée avec deux coordonnées, qui doivent par ailleurs être inférieures ou égales à la largeur et à la longueur de la fenêtre graphique où se passera la simulation.

Pour créer la liste d'agents, il est d'abord nécessaire de choisir le nombre d'agents que l'on souhaite avoir dans la liste. On utilisera, pour créer cette liste, le constructeur de la méthode Foule qui permet de créer une liste d'agents.

Ensuite, grâce à une boucle « while », on utilise le fait que la liste va se vider peu à peu lorsque les agents vont atteindre la sortie, et lorsqu'elle sera vide, le programme s'arrêtera. Dans cette boucle « while », on utilise toutes les classes créées permettant de dessiner, déplacer, supprimer les agents, etc.

Instance d'exécution

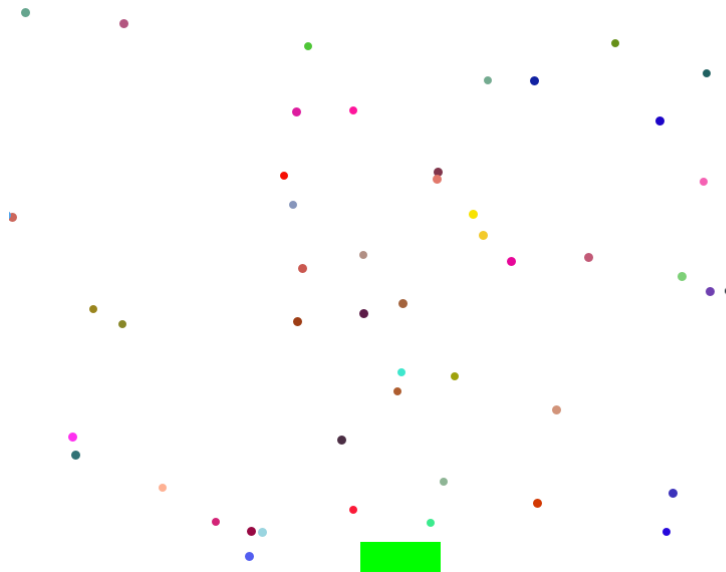


Figure 5 : Début de simulation

Cette première figure représente un début de simulation dans la fenêtre générée. On observe la sortie en vert fluo et tous les agents de différentes couleurs qui se dirigent vers la sortie.

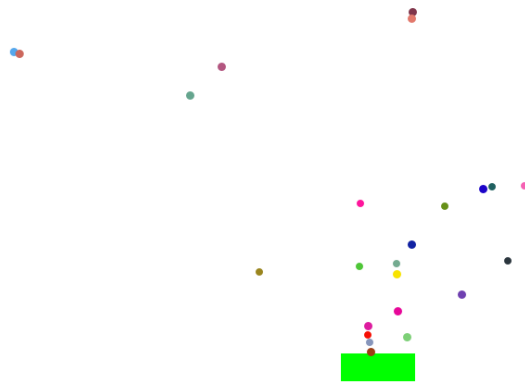


Figure 6 : Milieu de simulation

On se retrouve ici en milieu de simulation. On observe que certains agents, proches de la sortie, sont très proches les uns des autres. Il s'agit du fait que l'agent situé en arrière a baissé sa vitesse de déplacement. Il laisse celui devant lui avancer et avance quand il peut pour éviter la collision.

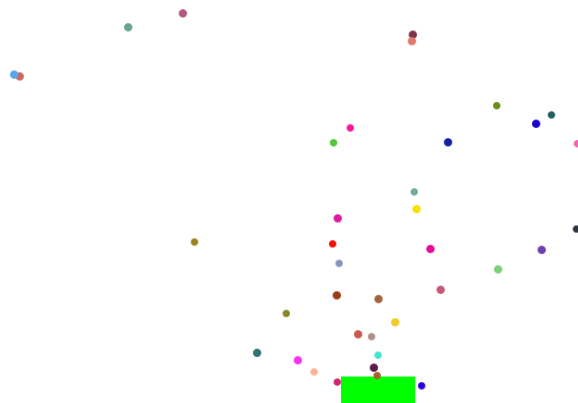


Figure 7 : Fin de simulation

On est ici en fin de simulation. On remarque une queue avec plusieurs agents les uns derrière les autres. En effet, ceux de derrière, comme dans la situation précédente, attendent que ceux de devant avance pour avancer et ne pas créer de collision.

Algorithmes utilisés

Il y a certains algorithmes qui ont demandé beaucoup plus de travail et de réflexion que d'autre. Dans cette optique, il est nécessaire d'en présenter le fonctionnement pour une meilleure compréhension globale du programme.

- Un premier algorithme assez simple, « Déplacer » :

Déplacer() :

$$NouvellePositionEnX = AnciennePositionEnX + Cap \times Vitesse$$

$$NouvellePositionEnY = AnciennePositionEnY + Cap \times Vitesse$$

Figure 8 : Méthode Déplacer

La seule difficulté pour ce programme est qu'il faut utiliser le cap et la vitesse pour déplacer l'agent car chaque agent n'a pas la même vitesse ni le même cap.

- Un algorithme pour les collisions, c'est-à-dire que les agents s'arrêtent lorsqu'ils rentrent en contact :

```

bloquer() :
    mélange de listeFoule

    nouvellesPositions = liste vide

    pour chaque agent a dans listeFoule :
        mettreAJourCap de A
        prochainePos = position de a + (cap de a * vitesse de a)
        nouvellesPositions.ajouter(prochainePos)

    peutAvancer = tableau booléen de la taille de listeFoule initialisation true

    pour i de 0 à la taille de listeFoule - 1 :
        pour j de 0 à la taille de listeFoule - 1 :
            si i différent de j :
                dist = norme(nouvellesPositions[i] - nouvellesPositions[j])
                minDist = rayon de listeFoule[i] + rayon de listeFoule[j]
                si dist < minDist :
                    peutAvancer[i] = false

    pour i de 0 à la taille de listeFoule - 1 :
        si peutAvancer[i] :
            listeFoule[i].position = nouvellesPositions[i]

    restants = liste vide

    pour chaque agent a dans listeFoule :
        si non a.AtteintObjectif() :
            restants.ajouter(a)

    listeFoule = restants

```

Figure 9 : Méthode Bloquer

- Un algorithme permettant d'éviter les collisions et de contourner les agents qui avance moins vite devant (deux algorithmes sont ici présentés car le deuxième utilise le premier pour fonctionner) :

```

deplacerDebloque(ListeAutresAgents, lambda, k) :
    dEtoile = (0,0)
    capObjectif = (objectif - position) / norme

    tentative = position + capObjectif * Vitesse
    directionOk = vrai

    Pour autre dans ListeAutresAgents
        Si autre différent de agent
            Calcul distance d entre les centres des agents
            Si d < somme des rayons
                directionOk = faux
    Si directionOk = vrai
        dEtoile = capObjectif
    Sinon
        Détour = pi/2
        Pour 1 <= i <= 12
            Angle = [-i*détour i*détour]
            Pour 0 <= j <= 1
                Dir = rotation(capObjectif, angle[j])
                futurPosition = PositionActuelle + Dir * Vitesse

                libre = vrai
                Pour agent dans ListeAutresAgents
                    Si autre différent de agent
                        Calcul distance entre future pos et PositionActuelle
                        Si dprime < somme rayons agents
                            Libre = faux

                Si libre = vrai
                    dEtoile = dir

    Si dEtoile = (0,0)
        L'agent s'arrête
    Cap = cap + (dEtoile - cap * Vitesse) / norme
    Position = position + cap * vitesse

```

Figure 10 : Méthode Débloquent (1/2)

```
debloquer (double lambda, int k) :  
    mélanger listeFoule  
  
    pour chaque agent a dans listeFoule :  
        a.deplacerDebloque(listeFoule, lambda, k)  
  
    restants = liste vide  
  
    pour chaque agent a dans listeFoule :  
        si non a.aAtteintObjectif() :  
            restants.ajouter(a)  
  
    listeFoule = restants
```

Figure 11 : Méthode Débloquer (2/2)

Tests

Pour en arriver à cette fin et avoir des agents qui s'évitent pour aller jusqu'à la sortie, plusieurs tests ont dû être effectués du début à la fin des essais de programmation.

- Le premier test était pour savoir quelle taille il fallait avoir pour la fenêtre. Une fenêtre trop petite ne peut pas accueillir assez d'agents pour avoir une expérience convenable et une fenêtre trop grande augmenterait le temps de la simulation inutilement, les agents ayant besoin de parcourir une trop longue distance pour aller jusqu'à la sortie.
- Après de simple test pour savoir si les agents bougeaient, quand ils pouvaient encore se déplacer les uns à travers les autres. Des tests sur le nombre d'agents ont été réalisé pour savoir ce que provoquait un trop grand nombre d'agents dans la fenêtre avec les collisions. Lorsque plusieurs centaines d'agents était envoyé d'agents, cela provoquait des paquets d'agents ne pouvant plus bouger. Lorsqu'on passait à quelque millier c'était une saturation de la fenêtre, plus aucun agent ne pouvait bouger vers la sortie.
- Pour les tests sur les évitements, il y a d'abord le même type de test qui a été réalisé que pour les collisions. Lorsque plusieurs centaines d'agents était envoyé d'agents, certains agents passaient à travers d'autres pendant toute la simulation. Lorsqu'on passait à quelque millier, beaucoup se traversaient mais la simulation aboutissait quand même. Aussi, des tests pour le nombre de rayons à partir duquel les agents calculent pour savoir s'ils évitent ou continue tout droit ont été réalisé. Plus le nombre de rayon est grand, mieux c'est car les agents peuvent mieux éviter.

Critiques et perspectives

Critiques

Sur la simulation, on peut remarquer que, lorsque le nombre augmente beaucoup, des agents se traversent dès le début de la simulation et pendant l'entièreté de celle-ci, ce qui n'est pas attendu.

Dès le début de ce projet on aurait pu mieux organiser notre programme pour avoir quelque chose de plus propre dès le début et mieux se répartir les tâches dès le début car le travail était effectué à deux sur la même chose pendant longtemps, ce qui nous a rendu moins efficace au début.

Perspectives

Avec plus de temps et d'énergie, il y aurait eu possibilité de pousser la programmation plus loin et d'aborder la sortie de train et le déplacement avec le gradient. Aussi, le programme aurait pu être beaucoup plus propre et structuré de manière parfaite.

Conclusion

L'intitulé du projet était très intéressant dès le début. Apprendre à programmer de cette manière est une nouveauté et c'est d'autant plus intéressant. Ce rapport a permis de montrer les idées développées et mise en pratique durant tout le projet et d'apporter un sens critique à la réalisation de ce projet. Le travail en équipe fut bienvenu car à certains moments il pouvait y avoir une baisse de motivation si une partie du programme ne fonctionnait pas. Même entre équipe il y avait de l'entraide et mélangé à un peu de compétition ce qui induisait une bonne ambiance. Peut-être qu'un peu plus d'explication sur certaines fonctionnalités de Java pourrait être nécessaire pour ne pas trop être perdu durant les TP car certaines personnes découvrent Java cette année et l'immersion dans ce logiciel de programmation n'est pas si simple.

Table des illustrations

Figure 1 : Diagramme UML de la classe Coordonnée -----	1
Figure 2 : Diagramme UML de la classe Agent -----	1
Figure 3 : Diagramme UML de la classe Foule-----	2
Figure 4 : Diagramme UML de l'ensemble des classes-----	2
Figure 5 : Début de simulation -----	3
Figure 6 : Milieu de simulation -----	4
Figure 7 : Fin de simulation -----	4
Figure 8 : Méthode Déplacer -----	5
Figure 9 : Méthode Bloquer-----	6
Figure 10 : Méthode Débloquent (1/2) -----	7
Figure 11 : Méthode Débloquent (2/2) -----	8